ДЗ 3. SQL поверх больших данных и инструменты визуализации БД

Формулировка задания

Цель: освоить принципы оптимизации хранения и обработки аналитических данных в экосистемах Big Data

Задание: взять датасет от прошлого задания или свой, не меньше 1 ГБ. Сохранить в двух вариантах: CSV и Parquet с партиционированием по дате. Выполнить три аналитических запроса поверх обоих вариантов. Зафиксировать время выполнения и объём прочитанных данных. Добавить фильтр по партиции и оставить только нужные колонки. Объяснить разницу через partition pruning и pushdown, приложив план запроса.

Форма отчётности: собрать дашборд из трёх-четырёх виджетов в любом BI-инструменте. В отчёте объяснить, за счёт чего запросы ускорились.

Критерии оценивания (max 6 б.): датасет ≥1 ГБ успешно сохранен в форматах CSV и Parquet с партиционированием по дате, выполнены три аналитических запроса с фиксацией времени и объема прочитанных данных для обоих вариантов, применены фильтр по партиции и проекция колонок с приложением планов запроса и объяснением механизмов partition pruning и pushdown, создан дашборд из 3-4 виджетов в BI-инструменте, отчёт содержит корректное объяснение причин ускорения запросов за счет оптимизаций хранения.

Решение

Инфраструктура

Стек технологий как на лекции - hive + trino + minio + superset.

Развернуто в docker'e:

```
docker compose up -d
docker ps
```

```
CONTAINER ID   IMAGE                                                STATUS                   PORTS
NAMES
04f1b312ef98   lr3-superset:5.0.0                                   Up 7 minutes (healthy)   0.0.0.0:8088->8088/
tcp, [::]:8088->8088/tcp                      lr3-superset
65b7751c7ebe   trinodb/trino:483                                    Up 7 minutes (healthy)   0.0.0.0:8080->8080/
tcp, [::]:8080->8080/tcp                      lr3-trino
88318349faa2   apache/hive:4.2.1                                    Up 7 minutes (healthy)   0.0.0.0:9083->9083/
tcp, [::]:9083->9083/tcp                      lr3-hive-metastore
080f9813f120   postgres:16                                          Up 8 minutes (healthy)   5432/tcp
lr3-superset-db
2f989e242e5d   ghcr.io/coollabsio/minio:RELEASE.2025-10-15T17-29-55Z   Up 8 minutes (healthy)   0.0.0.0:9000-9001->
9000-9001/tcp, [::]:9000-9001->9000-9001/tcp   lr3-minio
```

Загрузка данных

Для генерации данных выполнить скрипт:

```
python3 generate_data.py
```

Скрип генерирует csv размером ~1.1ГБ. Таблица широкая на 22 колонки, чтобы было видно преимущество колоночного хранения данных. Партиционирование по дням, события генерируются на 3 дня ⇒ 3 партиции.

Загружаем в s3.

```
docker compose up --force-recreate minio-init
```

Загружаем данные в trino.

```
docker exec -it lr3-trino trino
CREATE SCHEMA hive.hogwarts WITH (location = 's3a://warehouse/hogwarts/');

CREATE TABLE hive.hogwarts.events_csv (
    incident_id         varchar,
    created_at          varchar,
    created_date        varchar,
    student_name        varchar,
    house               varchar,
    school_year         varchar,
    professor           varchar,
    points_delta        varchar,
    reason              varchar,
    location            varchar,
    spell_cast          varchar,
    spell_success       varchar,
    wand_wood           varchar,
    wand_core           varchar,
    creature_involved   varchar,
    damage_galleons     varchar,
    detention_hours     varchar,
    witnesses_count     varchar,
    peeves_involved     varchar,
    filch_anger_level   varchar,
    owl_sent_to_parents varchar,
    howler_received     varchar
) WITH (
    format = 'CSV',
    external_location = 's3a://warehouse/raw/events_csv/',
    skip_header_line_count = 1
);

CREATE TABLE hive.hogwarts.events_parquet
WITH (
    format = 'PARQUET',
    partitioned_by = ARRAY['created_date']
) AS
SELECT
    CAST(incident_id AS bigint)           AS incident_id,
    CAST(created_at AS timestamp(3))      AS created_at,
    student_name,
    house,
    CAST(school_year AS integer)          AS school_year,
    professor,
    CAST(points_delta AS integer)         AS points_delta,
    reason,
    location,
    spell_cast,
    CAST(spell_success AS boolean)        AS spell_success,
    wand_wood,
    wand_core,
    creature_involved,
    CAST(damage_galleons AS decimal(10,2)) AS damage_galleons,
    CAST(detention_hours AS integer)      AS detention_hours,
    CAST(witnesses_count AS integer)      AS witnesses_count,
    CAST(peeves_involved AS boolean)      AS peeves_involved,
    CAST(filch_anger_level AS integer)    AS filch_anger_level,
    CAST(owl_sent_to_parents AS boolean)  AS owl_sent_to_parents,
    CAST(howler_received AS boolean)      AS howler_received,
    CAST(created_date AS date)            AS created_date
FROM hive.hogwarts.events_csv;

SELECT count(*) FROM hive.hogwarts.events_csv; -- 5400000
```

```
SELECT * FROM hive.hogwarts."events_parquet$partitions"; -- 3 partitions
SELECT created_date, count(*) FROM hive.hogwarts.events_parquet GROUP BY 1; -- 1.8M each
```

Запросы к данным

БД определяет домен баллов у факультетов во вселенной Гарри Поттера.

Результаты после первого дня

```
--- parquet
EXPLAIN
ANALYZE
SELECT house, sum(points_delta) AS points
FROM hive.hogwarts.events_parquet
WHERE created_date = DATE '2026-09-01'
GROUP BY house
ORDER BY points DESC;
```

```
Trino version: 483
Queued: 450.67us, Analysis: 6.77ms, Planning: 23.33ms, Execution: 187.94ms, Finishing: 0.00ns
Fragment 1 [SINGLE]
    CPU: 290.92us, Scheduled: 295.63us, Blocked 48.52ms (Input: 48.52ms, Output: 0.00ns), Input: 4 rows (94B); per
task: avg.: 4.00 std.dev.: 0.00, Output: 4 rows (94B)
    Peak Memory: 304B, Tasks count: 1; per task: max: 304B
    Output layout: [house, sum]
    Output partitioning: SINGLE []
    RemoteMerge[sourceFragmentIds = [2]]
        Layout: [house:varchar, sum:bigint]
        CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 49.00ms (2.40%), Output: 4 rows (94B)
        Input avg.: 4.00 rows, Input std.dev.: 0.00%

Fragment 2 [ROUND_ROBIN]
    CPU: 1.78ms, Scheduled: 1.91ms, Blocked 768.26ms (Input: 723.23ms, Output: 0.00ns), Input: 4 rows (94B); per ta
sk: avg.: 4.00 std.dev.: 0.00, Output: 4 rows (94B)
    Peak Memory: 304B, Tasks count: 1; per task: max: 304B
    Output layout: [house, sum]
    Output partitioning: SINGLE []
    LocalMerge[orderBy = [sum DESC NULLS LAST]]
    │   Layout: [house:varchar, sum:bigint]
    │   Estimates: {rows: 4 (94B), cpu: 0, memory: 0B, network: 0B}
    │   CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 45.00ms (2.20%), Output: 4 rows (94B)
    │   Input avg.: 0.25 rows, Input std.dev.: 300.00%
    └─ PartialSort[orderBy = [sum DESC NULLS LAST]]
       │   Layout: [house:varchar, sum:bigint]
       │   Estimates: {rows: 4 (94B), cpu: ?, memory: ?, network: ?}
       │   CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 0.00ns (0.00%), Output: 4 rows (94B)
       │   Input avg.: 0.25 rows, Input std.dev.: 300.00%
       └─ RemoteSource[sourceFragmentIds = [3]]
              Layout: [house:varchar, sum:bigint]
              CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 723.00ms (35.35%), Output: 4 rows (94B)
              Input avg.: 0.25 rows, Input std.dev.: 300.00%

Fragment 3 [HASH]
    CPU: 4.31ms, Scheduled: 4.68ms, Blocked 1.23s (Input: 620.13ms, Output: 0.00ns), Input: 12 rows (282B); per tas
k: avg.: 12.00 std.dev.: 0.00, Output: 4 rows (94B)
    Peak Memory: 632B, Tasks count: 1; per task: max: 632B
    Output layout: [house, sum]
    Output partitioning: ROUND_ROBIN []
    Aggregate[type = FINAL, keys = [house]]
    │   Layout: [house:varchar, sum:bigint]
    │   Estimates: {rows: 4 (94B), cpu: 40.38M, memory: 94B, network: 0B}
    │   CPU: 1.00ms (1.59%), Scheduled: 1.00ms (1.43%), Blocked: 0.00ns (0.00%), Output: 4 rows (94B)
    │   Input avg.: 0.75 rows, Input std.dev.: 173.21%
    │   sum := sum(sum_0)
    └─ LocalExchange[partitioning = HASH, arguments = [house::varchar]]
       │   Layout: [house:varchar, sum_0:bigint]
       │   Estimates: {rows: 1800000 (40.38MB), cpu: 40.38M, memory: 0B, network: 0B}
       │   CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 608.00ms (29.73%), Output: 12 rows (282B)
       │   Input avg.: 0.75 rows, Input std.dev.: 208.17%
       └─ RemoteSource[sourceFragmentIds = [4]]
              Layout: [house:varchar, sum_0:bigint]
              CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 620.00ms (30.32%), Output: 12 rows (282B)
              Input avg.: 0.75 rows, Input std.dev.: 208.17%

Fragment 4 [SOURCE]
    CPU: 61.85ms, Scheduled: 68.75ms, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 1800000 rows (40.34M
B); per task: avg.: 1800000.00 std.dev.: 0.00, Output: 12 rows (282B)
```

```
    Peak Memory: 400B, Tasks count: 1; per task: max: 800B
    Output layout: [house, sum_0]
    Output partitioning: HASH [house]
    Aggregate[type = PARTIAL, keys = [house]]
    │   Layout: [house:varchar, sum_0:bigint]
    │   Estimates: {rows: 1800000 (40.38MB), cpu: ?, memory: ?, network: ?}
    │   CPU: 11.00ms (17.46%), Scheduled: 11.00ms (15.71%), Blocked: 0.00ns (0.00%), Output: 12 rows (282B)
    │   Input avg.: 600000.00 rows, Input std.dev.: 63.18%
    │   sum_0 := sum(expr)
    └─ ScanProject[table = hive:hogwarts:events_parquet]
           Layout: [expr:bigint, house:varchar]
           Estimates: {rows: 1800000 (40.38MB), cpu: 33.51M, memory: 0B, network: 0B}/{rows: 1800000 (40.38MB), cp
u: 40.38M, memory: 0B, network: 0B}
           CPU: 51.00ms (80.95%), Scheduled: 58.00ms (82.86%), Blocked: 0.00ns (0.00%), Output: 1800000 rows (47.20
MB)
           Input avg.: 600000.00 rows, Input std.dev.: 63.18%
           expr := CAST(points_delta AS bigint)
           points_delta := points_delta:int:REGULAR
           house := house:string:REGULAR
           created_date:date:PARTITION_KEY
               :: [[2026-09-01]]
           Input: 1800000 rows (40.34MB), Filtered: 0.00%, Physical input: 5.81MB, Physical input time: 16.77ms, Sp
lits: 3, Splits generation wait time: 5.47ms
```

```sql
-- csv
EXPLAIN
ANALYZE
SELECT house, sum(CAST(points_delta AS integer)) AS points
FROM hive.hogwarts.events_csv
WHERE created_date = '2026-09-01'
GROUP BY house
ORDER BY points DESC;
```

```
Trino version: 483
Queued: 122.13us, Analysis: 6.08ms, Planning: 9.47ms, Execution: 301.45ms, Finishing: 0.00ns
Fragment 1 [SINGLE]
    CPU: 162.71us, Scheduled: 168.67us, Blocked 179.54ms (Input: 179.55ms, Output: 0.00ns), Input: 4 rows (94B); pe
r task: avg.: 4.00 std.dev.: 0.00, Output: 4 rows (94B)
    Peak Memory: 304B, Tasks count: 1; per task: max: 304B
    Output layout: [house, sum]
    Output partitioning: SINGLE []
    RemoteMerge[sourceFragmentIds = [2]]
        Layout: [house:varchar, sum:bigint]
        CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 180.00ms (2.06%), Output: 4 rows (94B)
        Input avg.: 4.00 rows, Input std.dev.: 0.00%

Fragment 2 [ROUND_ROBIN]
    CPU: 1.68ms, Scheduled: 1.83ms, Blocked 2.99s (Input: 2.82s, Output: 0.00ns), Input: 4 rows (94B); per task: av
g.: 4.00 std.dev.: 0.00, Output: 4 rows (94B)
    Peak Memory: 304B, Tasks count: 1; per task: max: 304B
    Output layout: [house, sum]
    Output partitioning: SINGLE []
    LocalMerge[orderBy = [sum DESC NULLS LAST]]
    │   Layout: [house:varchar, sum:bigint]
    │   Estimates: {rows: ? (?), cpu: 0, memory: 0B, network: 0B}
    │   CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 177.00ms (2.02%), Output: 4 rows (94B)
    │   Input avg.: 0.25 rows, Input std.dev.: 173.21%
    └─ PartialSort[orderBy = [sum DESC NULLS LAST]]
       │   Layout: [house:varchar, sum:bigint]
       │   CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 0.00ns (0.00%), Output: 4 rows (94B)
       │   Input avg.: 0.25 rows, Input std.dev.: 173.21%
       └─ RemoteSource[sourceFragmentIds = [3]]
              Layout: [house:varchar, sum:bigint]
              CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 2.82s (32.24%), Output: 4 rows (94B)
              Input avg.: 0.25 rows, Input std.dev.: 173.21%

Fragment 3 [HASH]
    CPU: 5.03ms, Scheduled: 6.71ms, Blocked 5.57s (Input: 2.78s, Output: 0.00ns), Input: 72 rows (1.65kB); per tas
k: avg.: 72.00 std.dev.: 0.00, Output: 4 rows (94B)
    Peak Memory: 848B, Tasks count: 1; per task: max: 848B
    Output layout: [house, sum]
    Output partitioning: ROUND_ROBIN []
    Aggregate[type = FINAL, keys = [house]]
    │   Layout: [house:varchar, sum:bigint]
    │   Estimates: {rows: ? (?), cpu: ?, memory: ?, network: 0B}
    │   CPU: 1.00ms (0.06%), Scheduled: 1.00ms (0.04%), Blocked: 0.00ns (0.00%), Output: 4 rows (94B)
    │   Input avg.: 4.50 rows, Input std.dev.: 173.21%
    │   sum := sum(sum_1)
    └─ LocalExchange[partitioning = HASH, arguments = [house::varchar]]
       │   Layout: [house:varchar, sum_1:bigint]
       │   Estimates: {rows: ? (?), cpu: ?, memory: 0B, network: 0B}
       │   CPU: 1.00ms (0.06%), Scheduled: 2.00ms (0.08%), Blocked: 2.79s (31.90%), Output: 72 rows (1.65kB)
       │   Input avg.: 4.50 rows, Input std.dev.: 53.29%
```

```
       └─ RemoteSource[sourceFragmentIds = [4]]
              Layout: [house:varchar, sum_1:bigint]
              CPU: 1.00ms (0.06%), Scheduled: 1.00ms (0.04%), Blocked: 2.78s (31.78%), Output: 72 rows (1.65kB)
              Input avg.: 4.50 rows, Input std.dev.: 53.29%

Fragment 4 [SOURCE]
    CPU: 1.68s, Scheduled: 2.47s, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 5400000 rows (187.03MB); p
er task: avg.: 5400000.00 std.dev.: 0.00, Output: 72 rows (1.65kB)
    Peak Memory: 1.77MB, Tasks count: 1; per task: max: 1.77MB
    Output layout: [house, sum_1]
    Output partitioning: HASH [house]
    Aggregate[type = PARTIAL, keys = [house]]
    │   Layout: [house:varchar, sum_1:bigint]
    │   CPU: 66.00ms (3.93%), Scheduled: 84.00ms (3.40%), Blocked: 0.00ns (0.00%), Output: 72 rows (1.65kB)
    │   Input avg.: 100000.00 rows, Input std.dev.: 12.69%
    │   sum_1 := sum(expr_0)
    └─ ScanFilterProject[table = hive:hogwarts:events_csv, filterPredicate = (created_date = varchar '2026-09-01')]
           Layout: [expr_0:bigint, house:varchar]
           Estimates: {rows: ? (?), cpu: ?, memory: 0B, network: 0B}/{rows: ? (?), cpu: ?, memory: 0B, network: 0
B}/{rows: ? (?), cpu: ?, memory: 0B, network: 0B}
           CPU: 1.61s (95.89%), Scheduled: 2.38s (96.43%), Blocked: 0.00ns (0.00%), Output: 1800000 rows (40.38MB)
           Input avg.: 300000.00 rows, Input std.dev.: 12.69%
           expr_0 := CAST(CAST(points_delta AS integer) AS bigint)
           created_date := created_date:string:REGULAR
           points_delta := points_delta:string:REGULAR
           house := house:string:REGULAR
           Input: 5400000 rows (187.03MB), Filtered: 66.67%, Physical input: 1.08GB, Physical input time: 1.05s, Sp
lits: 18, Splits generation wait time: 5.59ms
```

Анализ

Метрика	CSV	Parquet	Разница
Оператор скана	`ScanFilterProject`	`ScanProject`	фильтр исчез
Splits	18	3	6x
Прочитано строк	5 400 000	1 800 000	3x
Отфильтровано после чтения	66.67%	0%	—
Physical input	1.08 GB	5.81 MB	~185x
Physical input time	1.05 s	16.77 ms	~63x
CPU скана	1.68 s	61.85 ms	~27x
Execution (весь запрос)	301 ms	188 ms	~1.6x

Partition pruning. В плане Parquet нет `filterPredicate`: условие `created_date = DATE '2026-09-01'` превратилось в ограничение на ключ партиции (`created_date:date:PARTITION_KEY :: [[2026-09-01]]`). Trino выбрал нужную партицию по метаданным Hive Metastore ещё до чтения и создал splits только для файлов из `created_date=2026-09-01/`: 3 split'а и 1.8M строк, ничего не отфильтровано после чтения (`Filtered: 0.00%`). У CSV `created_date` - обычная колонка (`created_date:string:REGULAR`), поэтому читается весь файл: 18 split'ов, 5.4M строк. Фильтр применяется уже после чтения и отбрасывает 66.67% прочитанного.

Projection pushdown Запросу нужны только `house` и `points_delta`. Parquet хранит данные по колонкам, поэтому читаются только эти два column chunk'а из 22 колонок. Отсюда 5.81 MB, при том что вся партиция занимает ~24 MB. Дополнительно parquet жмет колонки через словари, RLE и bit packing + хранение в gzip. В csv вычитываются все колонки.

Типизация. В CSV все колонки имеют тип `varchar`, поэтому для каждой из 5.4M строк выполняется `CAST(CAST(points_delta AS integer) AS bigint)`. В Parquet значения уже хранятся как `int`, и разбирать текст не нужно. Это одна из причин разницы в CPU: 1.68 s против 62 ms.

Тк объем данных небольшой разница в общем времени исполнения тоже небольшая, тк много времени уходит на накладные расходы по планированию и запуску задачи.

Предвзятость Снейпа

Без фильтра по дате, чтобы увидеть эффект только от колоночного хранения.

```
--- parquet
EXPLAIN
ANALYZE
SELECT house, count(*) AS incidents, sum(points_delta) AS points, avg(points_delta) AS avg_points
FROM hive.hogwarts.events_parquet
WHERE professor = 'Severus Snape'
GROUP BY house
ORDER BY points;
```

```
Trino version: 483
Queued: 709.42us, Analysis: 10.39ms, Planning: 25.79ms, Execution: 193.93ms, Finishing: 0.00ns
Fragment 1 [SINGLE]
    CPU: 142.21us, Scheduled: 144.58us, Blocked 49.65ms (Input: 49.65ms, Output: 0.00ns), Input: 4 rows (166B); per
task: avg.: 4.00 std.dev.: 0.00, Output: 4 rows (166B)
    Peak Memory: 472B, Tasks count: 1; per task: max: 472B
    Output layout: [house, count, sum, avg]
    Output partitioning: SINGLE []
    RemoteMerge[sourceFragmentIds = [2]]
        Layout: [house:varchar, count:bigint, sum:bigint, avg:double]
        CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 50.00ms (2.19%), Output: 4 rows (166B)
        Input avg.: 4.00 rows, Input std.dev.: 0.00%

Fragment 2 [ROUND_ROBIN]
    CPU: 2.23ms, Scheduled: 2.67ms, Blocked 854.77ms (Input: 804.96ms, Output: 0.00ns), Input: 4 rows (166B); per t
ask: avg.: 4.00 std.dev.: 0.00, Output: 4 rows (166B)
    Peak Memory: 472B, Tasks count: 1; per task: max: 472B
    Output layout: [house, count, sum, avg]
    Output partitioning: SINGLE []
    LocalMerge[orderBy = [sum ASC NULLS LAST]]
    │   Layout: [house:varchar, count:bigint, sum:bigint, avg:double]
    │   Estimates: {rows: 4 (166B), cpu: 0, memory: 0B, network: 0B}
    │   CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 50.00ms (2.19%), Output: 4 rows (166B)
    │   Input avg.: 0.25 rows, Input std.dev.: 223.61%
    └─ PartialSort[orderBy = [sum ASC NULLS LAST]]
       │   Layout: [house:varchar, count:bigint, sum:bigint, avg:double]
       │   Estimates: {rows: 4 (166B), cpu: ?, memory: ?, network: ?}
       │   CPU: 2.00ms (1.06%), Scheduled: 2.00ms (0.82%), Blocked: 0.00ns (0.00%), Output: 4 rows (166B)
       │   Input avg.: 0.25 rows, Input std.dev.: 223.61%
       └─ RemoteSource[sourceFragmentIds = [3]]
              Layout: [house:varchar, count:bigint, sum:bigint, avg:double]
              CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 805.00ms (35.31%), Output: 4 rows (166B)
              Input avg.: 0.25 rows, Input std.dev.: 223.61%

Fragment 3 [HASH]
    CPU: 4.02ms, Scheduled: 4.44ms, Blocked 1.38s (Input: 696.65ms, Output: 0.00ns), Input: 36 rows (1.81kB); per t
ask: avg.: 36.00 std.dev.: 0.00, Output: 4 rows (166B)
    Peak Memory: 1.16kB, Tasks count: 1; per task: max: 1.16kB
    Output layout: [house, count, sum, avg]
    Output partitioning: ROUND_ROBIN []
    Aggregate[type = FINAL, keys = [house]]
    │   Layout: [house:varchar, count:bigint, sum:bigint, avg:double]
    │   Estimates: {rows: 4 (166B), cpu: 45.07M, memory: 166B, network: 0B}
    │   CPU: 1.00ms (0.53%), Scheduled: 1.00ms (0.41%), Blocked: 0.00ns (0.00%), Output: 4 rows (166B)
    │   Input avg.: 2.25 rows, Input std.dev.: 173.21%
    │   count := count(count_1)
    │   sum := sum(sum_2)
    │   avg := avg(avg_3)
    └─ LocalExchange[partitioning = HASH, arguments = [house::varchar]]
       │   Layout: [house:varchar, count_1:bigint, sum_2:bigint, avg_3:row(double, bigint)]
       │   Estimates: {rows: 540000 (45.07MB), cpu: 45.07M, memory: 0B, network: 0B}
       │   CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 678.00ms (29.74%), Output: 36 rows (1.81kB)
       │   Input avg.: 2.25 rows, Input std.dev.: 88.19%
       └─ RemoteSource[sourceFragmentIds = [4]]
              Layout: [house:varchar, count_1:bigint, sum_2:bigint, avg_3:row(double, bigint)]
              CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 697.00ms (30.57%), Output: 36 rows (1.81kB)
              Input avg.: 2.25 rows, Input std.dev.: 88.19%

Fragment 4 [SOURCE]
    CPU: 184.93ms, Scheduled: 239.15ms, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 5400000 rows (243.59
MB); per task: avg.: 5400000.00 std.dev.: 0.00, Output: 36 rows (1.95kB)
    Peak Memory: 19.33MB, Tasks count: 1; per task: max: 19.33MB
    Output layout: [house, count_1, sum_2, avg_3]
    Output partitioning: HASH [house]
    Aggregate[type = PARTIAL, keys = [house]]
    │   Layout: [house:varchar, count_1:bigint, sum_2:bigint, avg_3:row(double, bigint)]
    │   Estimates: {rows: 540000 (45.07MB), cpu: ?, memory: ?, network: ?}
    │   CPU: 22.00ms (11.70%), Scheduled: 22.00ms (9.05%), Blocked: 0.00ns (0.00%), Output: 36 rows (1.95kB)
```

```
    │   Input avg.: 149933.78 rows, Input std.dev.: 63.24%
    │   count_1 := count(*)
    │   sum_2 := sum(expr)
    │   avg_3 := avg(expr)
    └─ Project[]
       │   Layout: [expr:bigint, house:varchar]
       │   Estimates: {rows: 540000 (12.11MB), cpu: 12.11M, memory: 0B, network: 0B}
       │   CPU: 15.00ms (7.98%), Scheduled: 16.00ms (6.58%), Blocked: 0.00ns (0.00%), Output: 1349404 rows (35.37M
B)
       │   Input avg.: 149933.78 rows, Input std.dev.: 63.24%
       │   expr := CAST(points_delta AS bigint)
       └─ ScanFilterProject[table = hive:hogwarts:events_parquet, filterPredicate = (professor = varchar 'Severus S
nape')]
             Layout: [house:varchar, points_delta:integer]
             Estimates: {rows: 5400000 (100.54MB), cpu: 203.27M, memory: 0B, network: 0B}/{rows: 540000 (10.05MB),
cpu: 203.27M, memory: 0B, network: 0B}/{rows: 540000 (10.05MB), cpu: 10.05M, memory: 0B, network: 0B}
             CPU: 148.00ms (78.72%), Scheduled: 202.00ms (83.13%), Blocked: 0.00ns (0.00%), Output: 1349404 rows
(30.23MB)
             Input avg.: 600000.00 rows, Input std.dev.: 63.18%
             points_delta := points_delta:int:REGULAR
             house := house:string:REGULAR
             professor := professor:string:REGULAR
             created_date:date:PARTITION_KEY
                 :: [[2026-09-01], [2026-09-02], [2026-09-03]]
             Input: 5400000 rows (243.59MB), Filtered: 75.01%, Physical input: 17.42MB, Physical input time: 81.18
ms, Splits: 9, Splits generation wait time: 10.75ms
```

```sql
-- csv
EXPLAIN
ANALYZE
SELECT house,
       count(*)                          AS incidents,
       sum(CAST(points_delta AS integer)) AS points,
       avg(CAST(points_delta AS integer)) AS avg_points
FROM hive.hogwarts.events_csv
WHERE professor = 'Severus Snape'
GROUP BY house
ORDER BY points;
```

```
Trino version: 483
Queued: 148.46us, Analysis: 5.77ms, Planning: 12.32ms, Execution: 334.96ms, Finishing: 0.00ns
Fragment 1 [SINGLE]
    CPU: 184.17us, Scheduled: 187.08us, Blocked 211.50ms (Input: 211.50ms, Output: 0.00ns), Input: 4 rows (166B); p
er task: avg.: 4.00 std.dev.: 0.00, Output: 4 rows (166B)
    Peak Memory: 472B, Tasks count: 1; per task: max: 472B
    Output layout: [house, count, sum, avg]
    Output partitioning: SINGLE []
    RemoteMerge[sourceFragmentIds = [2]]
        Layout: [house:varchar, count:bigint, sum:bigint, avg:double]
        CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 212.00ms (2.04%), Output: 4 rows (166B)
        Input avg.: 4.00 rows, Input std.dev.: 0.00%

Fragment 2 [ROUND_ROBIN]
    CPU: 1.58ms, Scheduled: 1.62ms, Blocked 3.57s (Input: 3.36s, Output: 0.00ns), Input: 4 rows (166B); per task: a
vg.: 4.00 std.dev.: 0.00, Output: 4 rows (166B)
    Peak Memory: 472B, Tasks count: 1; per task: max: 472B
    Output layout: [house, count, sum, avg]
    Output partitioning: SINGLE []
    LocalMerge[orderBy = [sum ASC NULLS LAST]]
    │   Layout: [house:varchar, count:bigint, sum:bigint, avg:double]
    │   Estimates: {rows: ? (?), cpu: 0, memory: 0B, network: 0B}
    │   CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 210.00ms (2.02%), Output: 4 rows (166B)
    │   Input avg.: 0.25 rows, Input std.dev.: 173.21%
    └─ PartialSort[orderBy = [sum ASC NULLS LAST]]
       │   Layout: [house:varchar, count:bigint, sum:bigint, avg:double]
       │   CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 0.00ns (0.00%), Output: 4 rows (166B)
       │   Input avg.: 0.25 rows, Input std.dev.: 173.21%
       └─ RemoteSource[sourceFragmentIds = [3]]
             Layout: [house:varchar, count:bigint, sum:bigint, avg:double]
             CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 3.36s (32.27%), Output: 4 rows (166B)
             Input avg.: 0.25 rows, Input std.dev.: 173.21%

Fragment 3 [HASH]
    CPU: 6.52ms, Scheduled: 9.06ms, Blocked 6.63s (Input: 3.31s, Output: 0.00ns), Input: 72 rows (3.62kB); per tas
k: avg.: 72.00 std.dev.: 0.00, Output: 4 rows (166B)
    Peak Memory: 1.16kB, Tasks count: 1; per task: max: 1.16kB
    Output layout: [house, count, sum, avg]
    Output partitioning: ROUND_ROBIN []
    Aggregate[type = FINAL, keys = [house]]
    │   Layout: [house:varchar, count:bigint, sum:bigint, avg:double]
    │   Estimates: {rows: ? (?), cpu: ?, memory: ?, network: 0B}
    │   CPU: 1.00ms (0.05%), Scheduled: 1.00ms (0.03%), Blocked: 0.00ns (0.00%), Output: 4 rows (166B)
    │   Input avg.: 4.50 rows, Input std.dev.: 173.21%
```

```
     │   count := count(count_2)
     │   sum := sum(sum_3)
     │   avg := avg(avg_4)
     └─ LocalExchange[partitioning = HASH, arguments = [house::varchar]]
        │   Layout: [house:varchar, count_2:bigint, sum_3:bigint, avg_4:row(double, bigint)]
        │   Estimates: {rows: ? (?), cpu: ?, memory: 0B, network: 0B}
        │   CPU: 1.00ms (0.05%), Scheduled: 3.00ms (0.10%), Blocked: 3.32s (31.89%), Output: 72 rows (3.62kB)
        │   Input avg.: 4.50 rows, Input std.dev.: 76.17%
        └─ RemoteSource[sourceFragmentIds = [4]]
               Layout: [house:varchar, count_2:bigint, sum_3:bigint, avg_4:row(double, bigint)]
               CPU: 1.00ms (0.05%), Scheduled: 1.00ms (0.03%), Blocked: 3.31s (31.79%), Output: 72 rows (3.62kB)
               Input avg.: 4.50 rows, Input std.dev.: 76.17%

Fragment 4 [SOURCE]
    CPU: 1.99s, Scheduled: 3.11s, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 5400000 rows (212.52MB); p
er task: avg.: 5400000.00 std.dev.: 0.00, Output: 72 rows (3.90kB)
    Peak Memory: 50.34kB, Tasks count: 1; per task: max: 50.34kB
    Output layout: [house, count_2, sum_3, avg_4]
    Output partitioning: HASH [house]
    Aggregate[type = PARTIAL, keys = [house]]
    │   Layout: [house:varchar, count_2:bigint, sum_3:bigint, avg_4:row(double, bigint)]
    │   CPU: 61.00ms (3.06%), Scheduled: 85.00ms (2.73%), Blocked: 0.00ns (0.00%), Output: 72 rows (3.90kB)
    │   Input avg.: 74966.89 rows, Input std.dev.: 12.79%
    │   count_2 := count(*)
    │   sum_3 := sum(expr_0)
    │   avg_4 := avg(expr_0)
    └─ ScanFilterProject[table = hive:hogwarts:events_csv, filterPredicate = (professor = varchar 'Severus Snape')]
           Layout: [expr_0:bigint, house:varchar]
           Estimates: {rows: ? (?), cpu: ?, memory: 0B, network: 0B}/{rows: ? (?), cpu: ?, memory: 0B, network: 0
B}/{rows: ? (?), cpu: ?, memory: 0B, network: 0B}
           CPU: 1.93s (96.79%), Scheduled: 3.02s (97.11%), Blocked: 0.00ns (0.00%), Output: 1349404 rows (30.24MB)
           Input avg.: 300000.00 rows, Input std.dev.: 12.69%
           expr_0 := CAST(CAST(points_delta AS integer) AS bigint)
           points_delta := points_delta:string:REGULAR
           house := house:string:REGULAR
           professor := professor:string:REGULAR
           Input: 5400000 rows (212.52MB), Filtered: 75.01%, Physical input: 1.08GB, Physical input time: 1.54s, Sp
lits: 18, Splits generation wait time: 5.48ms
```

```
   house    | incidents |  points  | avg_points
------------+-----------+----------+------------
 Gryffindor |    336594 | -9248939 |     -27.48
 Hufflepuff |    338013 |    10103 |       0.03
 Ravenclaw  |    337964 |    14846 |       0.04
 Slytherin  |    336833 |  5220000 |       15.5
```

Анализ

Метрика	CSV	Parquet	Разница
Splits	18	9	2x
Прочитано строк	5 400 000	5 400 000	—
Physical input	1.08 GB	17.42 MB	~62x
Physical input time	1.54 s	81 ms	~19x
CPU скана	1.99 s	185 ms	~11x
Execution (весь запрос)	335 ms	194 ms	~1.7x

Pruning нет: Parquet читает все 3 партиции (`:: [[2026-09-01], [2026-09-02], [2026-09-03]]`) и все 5.4M строк, как и CSV. Вся разница в объёме - от **projection pushdown**: из 22 колонок читаются только `professor`, `house` и `points_delta`, сжатые словарём, RLE и gzip.

Фильтр `professor = 'Severus Snape'` у обоих форматов остался в `ScanFilterProject` и отбрасывает 75% строк.

Худший инцидент в каждой локации

Без фильтров, и запросу нужны все 22 колонки: `max_by(ROW(...), damage_galleons)` возвращает полную запись самого дорогого инцидента. Демонстрирует преимущество parquet в сжатии данных +

типизации колонок.

```
-- parquet
EXPLAIN
ANALYZE
SELECT location,
       count(*)                AS incidents,
       sum(damage_galleons)    AS damage,
       max_by(ROW(incident_id, created_at, created_date, student_name, house, school_year, professor,
                  points_delta, reason, spell_cast, spell_success, wand_wood, wand_core, creature_involved,
                  detention_hours, witnesses_count, peeves_involved, filch_anger_level,
                  owl_sent_to_parents, howler_received),
              damage_galleons) AS worst_incident
FROM hive.hogwarts.events_parquet
GROUP BY location
ORDER BY damage DESC;
```

```
Trino version: 483
Queued: 502.50us, Analysis: 8.95ms, Planning: 19.15ms, Execution: 1.51s, Finishing: 0.00ns
Fragment 1 [SINGLE]
    CPU: 221.63us, Scheduled: 227.17us, Blocked 1.37s (Input: 1.37s, Output: 0.00ns), Input: 16 rows (3.84kB); per
task: avg.: 16.00 std.dev.: 0.00, Output: 16 rows (3.84kB)
    Peak Memory: 4.67kB, Tasks count: 1; per task: max: 4.67kB
    Output layout: [location, count, sum, max_by]
    Output partitioning: SINGLE []
    RemoteMerge[sourceFragmentIds = [2]]
        Layout: [location:varchar, count:bigint, sum:decimal(38,2), max_by:row(bigint, timestamp(3), date, varchar,
varchar, integer, varchar, integer, varchar, varchar, boolean, varchar, varchar, varchar, integer, integer, boolea
n, integer, boolean, boolean)]
        CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 1.37s (2.00%), Output: 16 rows (3.84kB)
        Input avg.: 16.00 rows, Input std.dev.: 0.00%

Fragment 2 [ROUND_ROBIN]
    CPU: 2.73ms, Scheduled: 2.97ms, Blocked 23.30s (Input: 21.93s, Output: 0.00ns), Input: 16 rows (3.84kB); per ta
sk: avg.: 16.00 std.dev.: 0.00, Output: 16 rows (3.84kB)
    Peak Memory: 4.67kB, Tasks count: 1; per task: max: 4.67kB
    Output layout: [location, count, sum, max_by]
    Output partitioning: SINGLE []
    LocalMerge[orderBy = [sum DESC NULLS LAST]]
    │   Layout: [location:varchar, count:bigint, sum:decimal(38,2), max_by:row(bigint, timestamp(3), date, varchar,
varchar, integer, varchar, integer, varchar, varchar, boolean, varchar, varchar, varchar, integer, integer, boolea
n, integer, boolean, boolean)]
    │   Estimates: {rows: 16 (1.59kB), cpu: 0, memory: 0B, network: 0B}
    │   CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 1.37s (2.00%), Output: 16 rows (3.84kB)
    │   Input avg.: 1.00 rows, Input std.dev.: 136.93%
    └─ PartialSort[orderBy = [sum DESC NULLS LAST]]
       │   Layout: [location:varchar, count:bigint, sum:decimal(38,2), max_by:row(bigint, timestamp(3), date, varch
ar, varchar, integer, varchar, integer, varchar, varchar, boolean, varchar, varchar, varchar, integer, integer, boo
lean, integer, boolean, boolean)]
       │   Estimates: {rows: 16 (1.59kB), cpu: ?, memory: ?, network: ?}
       │   CPU: 1.00ms (0.01%), Scheduled: 1.00ms (0.01%), Blocked: 0.00ns (0.00%), Output: 16 rows (3.84kB)
       │   Input avg.: 1.00 rows, Input std.dev.: 136.93%
       └─ RemoteSource[sourceFragmentIds = [3]]
              Layout: [location:varchar, count:bigint, sum:decimal(38,2), max_by:row(bigint, timestamp(3), date, va
rchar, varchar, integer, varchar, integer, varchar, varchar, boolean, varchar, varchar, varchar, integer, integer,
boolean, integer, boolean, boolean)]
              CPU: 0.00ns (0.00%), Scheduled: 1.00ms (0.01%), Blocked: 21.93s (32.00%), Output: 16 rows (3.84kB)
              Input avg.: 1.00 rows, Input std.dev.: 136.93%

Fragment 3 [HASH]
    CPU: 15.94ms, Scheduled: 18.40ms, Blocked 43.85s (Input: 21.94s, Output: 0.00ns), Input: 976 rows (247.10kB); p
er task: avg.: 976.00 std.dev.: 0.00, Output: 16 rows (3.84kB)
    Peak Memory: 1.42MB, Tasks count: 1; per task: max: 1.42MB
    Output layout: [location, count, sum, max_by]
    Output partitioning: ROUND_ROBIN []
    Aggregate[type = FINAL, keys = [location]]
    │   Layout: [location:varchar, count:bigint, sum:decimal(38,2), max_by:row(bigint, timestamp(3), date, varchar,
varchar, integer, varchar, integer, varchar, varchar, boolean, varchar, varchar, varchar, integer, integer, boolea
n, integer, boolean, boolean)]
    │   Estimates: {rows: 16 (1.59kB), cpu: 721.29M, memory: 1.59kB, network: 0B}
    │   CPU: 12.00ms (0.17%), Scheduled: 15.00ms (0.20%), Blocked: 0.00ns (0.00%), Output: 16 rows (3.84kB)
    │   Input avg.: 61.00 rows, Input std.dev.: 127.48%
    │   count := count(count_0)
    │   sum := sum(sum_1)
    │   max_by := max_by(max_by_2)
    └─ LocalExchange[partitioning = HASH, arguments = [location::varchar]]
       │   Layout: [location:varchar, count_0:bigint, sum_1:varbinary, max_by_2:row(decimal(10,2), row(bigint, time
stamp(3), date, varchar, varchar, integer, varchar, integer, varchar, varchar, boolean, varchar, varchar, varchar,
integer, integer, boolean, integer, boolean, boolean))]
       │   Estimates: {rows: 5400000 (721.29MB), cpu: 721.29M, memory: 0B, network: 0B}
       │   CPU: 1.00ms (0.01%), Scheduled: 2.00ms (0.03%), Blocked: 21.92s (31.99%), Output: 976 rows (247.10kB)
       │   Input avg.: 61.00 rows, Input std.dev.: 119.56%
```

```
       └─ RemoteSource[sourceFragmentIds = [4]]
             Layout: [location:varchar, count_0:bigint, sum_1:varbinary, max_by_2:row(decimal(10,2), row(bigint, t
imestamp(3), date, varchar, varchar, integer, varchar, integer, varchar, varchar, boolean, varchar, varchar, varcha
r, integer, integer, boolean, integer, boolean, boolean))]
             CPU: 1.00ms (0.01%), Scheduled: 1.00ms (0.01%), Blocked: 21.94s (32.02%), Output: 976 rows (247.10kB)
             Input avg.: 61.00 rows, Input std.dev.: 119.56%

Fragment 4 [SOURCE]
    CPU: 7.25s, Scheduled: 7.62s, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 5400000 rows (1.34GB); per
task: avg.: 5400000.00 std.dev.: 0.00, Output: 976 rows (254.72kB)
    Peak Memory: 62.48MB, Tasks count: 1; per task: max: 62.48MB
    Output layout: [location, count_0, sum_1, max_by_2]
    Output partitioning: HASH [location]
    Aggregate[type = PARTIAL, keys = [location]]
    │   Layout: [location:varchar, count_0:bigint, sum_1:varbinary, max_by_2:row(decimal(10,2), row(bigint, timesta
mp(3), date, varchar, varchar, integer, varchar, integer, varchar, varchar, boolean, varchar, varchar, varchar, int
eger, integer, boolean, integer, boolean, boolean))]
    │   Estimates: {rows: 5400000 (721.29MB), cpu: ?, memory: ?, network: ?}
    │   CPU: 73.00ms (1.00%), Scheduled: 72.00ms (0.94%), Blocked: 0.00ns (0.00%), Output: 976 rows (254.72kB)
    │   Input avg.: 600000.00 rows, Input std.dev.: 63.18%
    │   count_0 := count(*)
    │   sum_1 := sum(damage_galleons)
    │   max_by_2 := max_by(expr, damage_galleons)
    └─ ScanProject[table = hive:hogwarts:events_parquet]
           Layout: [damage_galleons:decimal(10,2), expr:row(bigint, timestamp(3), date, varchar, varchar, integer,
varchar, integer, varchar, varchar, boolean, varchar, varchar, varchar, integer, integer, boolean, integer, boolea
n, boolean), location:varchar]
           Estimates: {rows: 5400000 (438.05MB), cpu: 1.18G, memory: 0B, network: 0B}/{rows: 5400000 (438.05MB), cp
u: 438.05M, memory: 0B, network: 0B}
           CPU: 7.18s (98.79%), Scheduled: 7.55s (98.80%), Blocked: 0.00ns (0.00%), Output: 5400000 rows (1.21GB)
           Input avg.: 600000.00 rows, Input std.dev.: 63.18%
           expr := ROW (incident_id, created_at, created_date, student_name, house, school_year, professor, points_
delta, reason, spell_cast, spell_success, wand_wood, wand_core, creature_involved, detention_hours, witnesses_coun
t, peeves_involved, filch_anger_level, owl_sent_to_parents, howler_received)
           detention_hours := detention_hours:int:REGULAR
           location := location:string:REGULAR
           damage_galleons := damage_galleons:decimal(10,2):REGULAR
           wand_wood := wand_wood:string:REGULAR
           created_date := created_date:date:PARTITION_KEY
               :: [[2026-09-01], [2026-09-02], [2026-09-03]]
           creature_involved := creature_involved:string:REGULAR
           peeves_involved := peeves_involved:boolean:REGULAR
           wand_core := wand_core:string:REGULAR
           owl_sent_to_parents := owl_sent_to_parents:boolean:REGULAR
           incident_id := incident_id:bigint:REGULAR
           spell_success := spell_success:boolean:REGULAR
           howler_received := howler_received:boolean:REGULAR
           witnesses_count := witnesses_count:int:REGULAR
           points_delta := points_delta:int:REGULAR
           spell_cast := spell_cast:string:REGULAR
           student_name := student_name:string:REGULAR
           created_at := created_at:timestamp:REGULAR
           house := house:string:REGULAR
           school_year := school_year:int:REGULAR
           professor := professor:string:REGULAR
           filch_anger_level := filch_anger_level:int:REGULAR
           reason := reason:string:REGULAR
           Input: 5400000 rows (1.34GB), Filtered: 0.00%, Physical input: 73.37MB, Physical input time: 107.18ms, S
plits: 9, Splits generation wait time: 8.92ms
```

```csv
--- csv
EXPLAIN
ANALYZE
SELECT location,
       count(*)                                          AS incidents,
       sum(CAST(damage_galleons AS decimal(12, 2)))      AS damage,
       max_by(ROW(incident_id, created_at, created_date, student_name, house, school_year, professor,
                  points_delta, reason, spell_cast, spell_success, wand_wood, wand_core, creature_involved,
                  detention_hours, witnesses_count, peeves_involved, filch_anger_level,
                  owl_sent_to_parents, howler_received),
              CAST(damage_galleons AS decimal(12, 2))) AS worst_incident
FROM hive.hogwarts.events_csv
GROUP BY location
ORDER BY damage DESC;
```

```
Trino version: 483
Queued: 134.67us, Analysis: 6.00ms, Planning: 7.25ms, Execution: 1.15s, Finishing: 0.00ns
Fragment 1 [SINGLE]
    CPU: 359.42us, Scheduled: 363.75us, Blocked 1.03s (Input: 1.03s, Output: 0.00ns), Input: 16 rows (4.86kB); per
task: avg.: 16.00 std.dev.: 0.00, Output: 16 rows (4.86kB)
    Peak Memory: 5.90kB, Tasks count: 1; per task: max: 5.90kB
    Output layout: [location, count, sum, max_by]
    Output partitioning: SINGLE []
```

```
    RemoteMerge[sourceFragmentIds = [2]]
        Layout: [location:varchar, count:bigint, sum:decimal(38,2), max_by:row(varchar, varchar, varchar, varchar,
varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varcha
r, varchar, varchar, varchar)]
        CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 1.03s (2.01%), Output: 16 rows (4.86kB)
        Input avg.: 16.00 rows, Input std.dev.: 0.00%

Fragment 2 [ROUND_ROBIN]
    CPU: 2.92ms, Scheduled: 3.65ms, Blocked 17.46s (Input: 16.44s, Output: 0.00ns), Input: 16 rows (4.86kB); per ta
sk: avg.: 16.00 std.dev.: 0.00, Output: 16 rows (4.86kB)
    Peak Memory: 5.90kB, Tasks count: 1; per task: max: 5.90kB
    Output layout: [location, count, sum, max_by]
    Output partitioning: SINGLE []
    LocalMerge[orderBy = [sum DESC NULLS LAST]]
    │   Layout: [location:varchar, count:bigint, sum:decimal(38,2), max_by:row(varchar, varchar, varchar, varchar,
varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varcha
r, varchar, varchar, varchar)]
    │   Estimates: {rows: ? (?), cpu: 0, memory: 0B, network: 0B}
    │   CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 1.03s (2.01%), Output: 16 rows (4.86kB)
    │   Input avg.: 1.00 rows, Input std.dev.: 217.94%
    └─ PartialSort[orderBy = [sum DESC NULLS LAST]]
       │   Layout: [location:varchar, count:bigint, sum:decimal(38,2), max_by:row(varchar, varchar, varchar, varcha
r, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varc
har, varchar, varchar, varchar)]
       │   CPU: 2.00ms (0.02%), Scheduled: 3.00ms (0.02%), Blocked: 0.00ns (0.00%), Output: 16 rows (4.86kB)
       │   Input avg.: 1.00 rows, Input std.dev.: 217.94%
       └─ RemoteSource[sourceFragmentIds = [3]]
              Layout: [location:varchar, count:bigint, sum:decimal(38,2), max_by:row(varchar, varchar, varchar, var
char, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, v
archar, varchar, varchar, varchar)]
              CPU: 0.00ns (0.00%), Scheduled: 0.00ns (0.00%), Blocked: 16.44s (32.09%), Output: 16 rows (4.86kB)
              Input avg.: 1.00 rows, Input std.dev.: 217.94%

Fragment 3 [HASH]
    CPU: 14.07ms, Scheduled: 20.85ms, Blocked 32.72s (Input: 16.37s, Output: 0.00ns), Input: 320 rows (101.43kB); p
er task: avg.: 320.00 std.dev.: 0.00, Output: 16 rows (4.86kB)
    Peak Memory: 1.42MB, Tasks count: 1; per task: max: 1.42MB
    Output layout: [location, count, sum, max_by]
    Output partitioning: ROUND_ROBIN []
    Aggregate[type = FINAL, keys = [location]]
    │   Layout: [location:varchar, count:bigint, sum:decimal(38,2), max_by:row(varchar, varchar, varchar, varchar,
varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varcha
r, varchar, varchar, varchar)]
    │   Estimates: {rows: ? (?), cpu: ?, memory: ?, network: 0B}
    │   CPU: 7.00ms (0.06%), Scheduled: 9.00ms (0.06%), Blocked: 0.00ns (0.00%), Output: 16 rows (4.86kB)
    │   Input avg.: 20.00 rows, Input std.dev.: 127.48%
    │   count := count(count_1)
    │   sum := sum(sum_2)
    │   max_by := max_by(max_by_3)
    └─ LocalExchange[partitioning = HASH, arguments = [location::varchar]]
       │   Layout: [location:varchar, count_1:bigint, sum_2:varbinary, max_by_3:row(decimal(12,2), row(varchar, var
char, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, v
archar, varchar, varchar, varchar, varchar, varchar))]
       │   Estimates: {rows: ? (?), cpu: ?, memory: 0B, network: 0B}
       │   CPU: 4.00ms (0.03%), Scheduled: 9.00ms (0.06%), Blocked: 16.36s (31.93%), Output: 320 rows (101.43kB)
       │   Input avg.: 20.00 rows, Input std.dev.: 34.64%
       └─ RemoteSource[sourceFragmentIds = [4]]
              Layout: [location:varchar, count_1:bigint, sum_2:varbinary, max_by_3:row(decimal(12,2), row(varchar,
varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varcha
r, varchar, varchar, varchar, varchar, varchar, varchar))]
              CPU: 1.00ms (0.01%), Scheduled: 1.00ms (0.01%), Blocked: 16.37s (31.95%), Output: 320 rows (101.43kB)
              Input avg.: 20.00 rows, Input std.dev.: 34.64%

Fragment 4 [SOURCE]
    CPU: 11.76s, Scheduled: 15.26s, Blocked 0.00ns (Input: 0.00ns, Output: 0.00ns), Input: 5400000 rows (1.51GB); p
er task: avg.: 5400000.00 std.dev.: 0.00, Output: 320 rows (103.93kB)
    Peak Memory: 31.68MB, Tasks count: 1; per task: max: 31.68MB
    Output layout: [location, count_1, sum_2, max_by_3]
    Output partitioning: HASH [location]
    Aggregate[type = PARTIAL, keys = [location]]
    │   Layout: [location:varchar, count_1:bigint, sum_2:varbinary, max_by_3:row(decimal(12,2), row(varchar, varcha
r, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varc
har, varchar, varchar, varchar, varchar, varchar))]
    │   CPU: 315.00ms (2.67%), Scheduled: 389.00ms (2.55%), Blocked: 0.00ns (0.00%), Output: 320 rows (103.93kB)
    │   Input avg.: 300000.00 rows, Input std.dev.: 12.69%
    │   count_1 := count(*)
    │   sum_2 := sum(expr)
    │   max_by_3 := max_by(expr_0, expr)
    └─ ScanProject[table = hive:hogwarts:events_csv]
           Layout: [expr:decimal(12,2), expr_0:row(varchar, varchar, varchar, varchar, varchar, varchar, varchar, v
archar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varchar, varcha
r), location:varchar]
           Estimates: {rows: ? (?), cpu: ?, memory: 0B, network: 0B}/{rows: ? (?), cpu: ?, memory: 0B, network: 0B}
           CPU: 11.45s (97.21%), Scheduled: 14.87s (97.31%), Blocked: 0.00ns (0.00%), Output: 5400000 rows (1.52GB)
```

```
            Input avg.: 300000.00 rows, Input std.dev.: 12.69%
            expr := CAST(damage_galleons AS decimal(12,2))
            expr_0 := ROW (incident_id, created_at, created_date, student_name, house, school_year, professor, point
s_delta, reason, spell_cast, spell_success, wand_wood, wand_core, creature_involved, detention_hours, witnesses_cou
nt, peeves_involved, filch_anger_level, owl_sent_to_parents, howler_received)
            created_at := created_at:string:REGULAR
            created_date := created_date:string:REGULAR
            location := location:string:REGULAR
            incident_id := incident_id:string:REGULAR
            wand_wood := wand_wood:string:REGULAR
            creature_involved := creature_involved:string:REGULAR
            wand_core := wand_core:string:REGULAR
            witnesses_count := witnesses_count:string:REGULAR
            howler_received := howler_received:string:REGULAR
            filch_anger_level := filch_anger_level:string:REGULAR
            spell_success := spell_success:string:REGULAR
            damage_galleons := damage_galleons:string:REGULAR
            peeves_involved := peeves_involved:string:REGULAR
            detention_hours := detention_hours:string:REGULAR
            spell_cast := spell_cast:string:REGULAR
            points_delta := points_delta:string:REGULAR
            owl_sent_to_parents := owl_sent_to_parents:string:REGULAR
            student_name := student_name:string:REGULAR
            house := house:string:REGULAR
            school_year := school_year:string:REGULAR
            professor := professor:string:REGULAR
            reason := reason:string:REGULAR
            Input: 5400000 rows (1.51GB), Filtered: 0.00%, Physical input: 1.08GB, Physical input time: 1.28s, Split
s: 18, Splits generation wait time: 6.41ms
```

Топ-5 локаций по ущербу (без колонки `worst_incident`):

```
        location        | incidents |   damage
------------------------+-----------+-------------
 Quidditch Pitch        |    338194 | 11967945.02
 Gryffindor Common Room |    337231 | 11936398.90
 Hagrid's Hut           |    337968 | 11915972.67
 Room of Requirement    |    337573 | 11905139.73
 Potions Dungeon        |    337406 | 11904938.60
(5 rows)
```

Анализ

Метрика	CSV	Parquet	Разница
Splits	18	9	2x
Прочитано строк	5 400 000	5 400 000	—
Physical input	1.08 GB	73.37 MB	~15x
Physical input time	1.28 s	107 ms	~12x
CPU скана (суммарно)	11.76 s	7.25 s	~1.6x
Execution (весь запрос)	1.15 s	1.51 s	Parquet медленнее

Parquet прочитал 73.37 MB - это весь размер таблицы, то есть все колонки всех партиций. Разница в 15X по объёму получается только за счёт хранения: бинарные типы вместо текста, словари, RLE / bit packing и gzip.

При этом общее время у Parquet больше, предположительно из-за

- Параллелизма - trino запущено на 16 ядрах, csv файл режется на 18 split'ов, parquet на 9 ⇒ у csv больше параллелизм.
- Чтение относительно дешевое - minio работает локально, данные не передаются по сети, чтение с ssd быстрое.

При увеличении объема и переходе в распределенное хранилища разрыв уйдет.

Вывод Колоночный формат даёт основной выигрыш, когда запрос читает часть колонок (Q1, Q2). Если нужны все колонки, остаётся только выигрыш в объёме хранения и чтения.

За счёт чего ускорились запросы

Механизм	Q1	Q2	Q3	Эффект
Partition pruning	+	-	-	читаются только нужные партиции: 3 split'а из 9, 1.8M строк
Projection pushdown	+	+	-	читаются только нужные колонки: 5.8 / 17.4 MB вместо 1.08 GB
Сжатие и кодирование (словари, RLE, bit packing, gzip)	+	+	+	вся таблица 73 MB вместо 1.1 GB (~15x)
Типизация колонок	+	+	+	нет парсинга текста и `CAST` на каждой строке
Predicate pushdown	+	-	-	в Q2 фильтр передан в reader, но row group'ы не пропущены: данные не отсортированы по `professor`

Создание дэшбордов

Superset UI по ссылке - `http://localhost:8088`.

Подключаем trino к superset:

```
docker exec lr3-superset superset set-database-uri -d trino -u trino://admin@trino:8080/hive
```

Проверяем подключение в UI

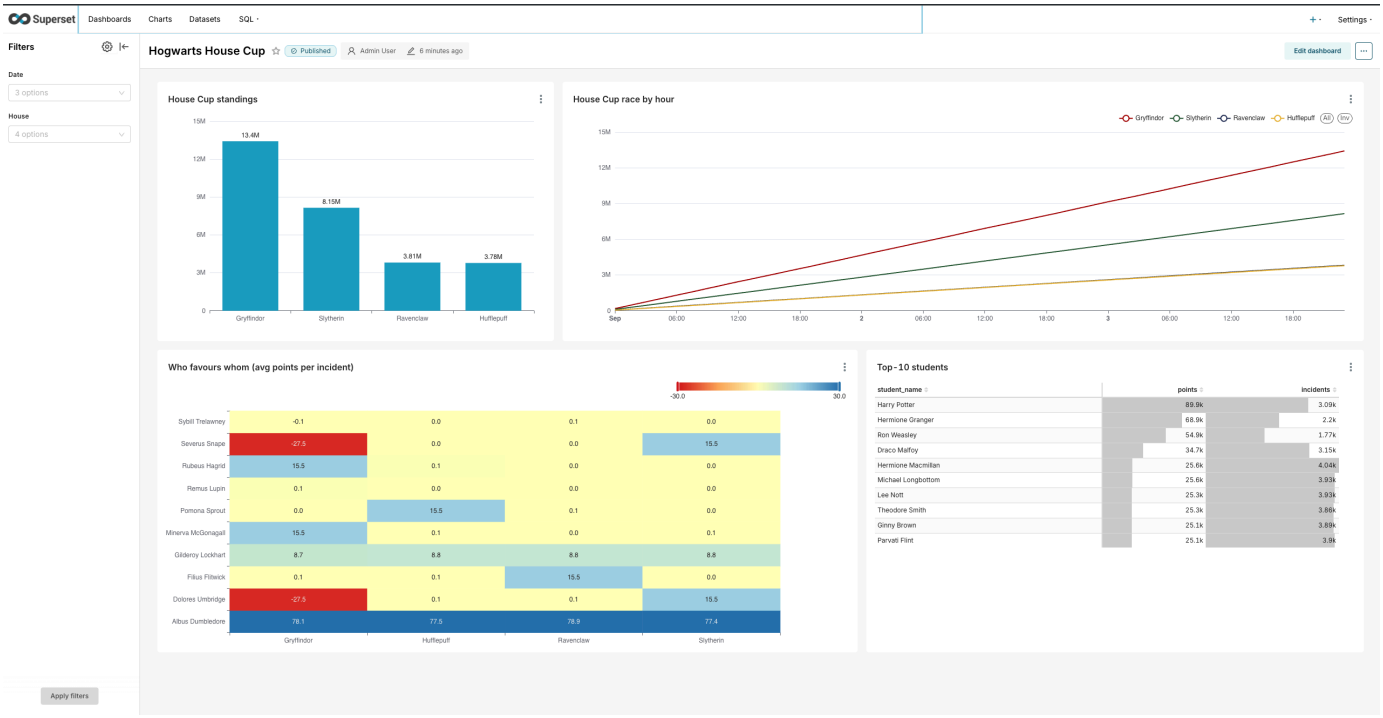
```
select count(*)
from hogwarts.events_csv; -- 5400000
select count(*)
from hogwarts.events_parquet; -- 5400000
```

Добавлено 4 виджета

- Результаты чемпионата
- Динамика чемпионата
- Предвзятость преподавателей
- Рейтинг студентов

Доступны 2 фильтра:

- День соревнования
- Факультет (для рейтинга студентов)



Импорт

Дэшборд экспортирован в superset/hogwarts_dashboard.zip

Импорт:

```
docker cp superset/hogwarts_dashboard.zip lr3-superset:/tmp/
docker exec lr3-superset superset import-dashboards -p /tmp/hogwarts_dashboard.zip -u admin
```

Или через UI: Dashboards => Import dashboards => выбрать zip